

PROGRAM 8

Implement Iterator pattern in java.

Problem Description: The students must demonstrate Iterator design pattern to traverse an array list and a tree using the same interface.

Method:

1. Decoupling: The pattern decouples clients from the implementation of the aggregate object, allowing for changes to the internal data structure without affecting clients that use the iterator.
2. Encapsulation: The iterator encapsulates the traversal logic, hiding the complexity of iterating over the collection from clients.
3. Flexible Traversal: The pattern supports different traversal algorithms, such as forward, backward, or random iteration.
4. Reusable Interface: The iterator interface provides a consistent way to iterate over different types of collections, promoting code reuse and maintainability.

CODE:

```
import java.util.*;
```

```
// Main class
```

```
public class IteratorPatternDemo {
```

```
    // Iterator Interface
```

```
    interface MyIterator {
```

```
        boolean hasNext();
```

```
        Object next();
```

```
    }
```

```
    // Aggregate Interface
```

```
    interface Container {
```

```

        MyIterator createIterator();
    }

// Concrete Aggregate 1: ArrayList
static class NameCollection implements Container {
    private List<String> names = new ArrayList<>();

    public void addName(String name) {
        names.add(name);
    }

    public MyIterator createIterator() {
        return new NameIterator();
    }

// Concrete Iterator for ArrayList
private class NameIterator implements MyIterator {
    int index = 0;

    public boolean hasNext() {
        return index < names.size();
    }

    public Object next() {
        if (this.hasNext()) {
            return names.get(index++);
        }
        return null;
    }
}
}

```

```
// Tree Node
```

```
static class TreeNode {
```

```
    String value;
```

```
    List<TreeNode> children = new ArrayList<>();
```

```
    TreeNode(String value) {
```

```
        this.value = value;
```

```
    }
```

```
    void addChild(TreeNode node) {
```

```
        children.add(node);
```

```
    }
```

```
}
```

```
// Concrete Aggregate 2: Tree
```

```
static class TreeCollection implements Container {
```

```
    private TreeNode root;
```

```
    public TreeCollection(TreeNode root) {
```

```
        this.root = root;
```

```
    }
```

```
    public MyIterator createIterator() {
```

```
        return new TreeIterator(root);
```

```
    }
```

```
// Concrete Iterator for Tree (DFS traversal)
```

```
private class TreeIterator implements MyIterator {
```

```
    private Stack<TreeNode> stack = new Stack<>();
```

```

public TreeIterator(TreeNode root) {
    if (root != null) stack.push(root);
}

public boolean hasNext() {
    return !stack.isEmpty();
}

public Object next() {
    if (!hasNext()) return null;

    TreeNode current = stack.pop();

    // push children in reverse for correct order
    List<TreeNode> children = current.children;
    for (int i = children.size() - 1; i >= 0; i--) {
        stack.push(children.get(i));
    }

    return current.value;
}
}

// Main method
public static void main(String[] args) {

    // ArrayList traversal
    NameCollection names = new NameCollection();
    names.addName("Alice");
    names.addName("Bob");
}

```

```
names.addName("Charlie");

System.out.println("Traversing ArrayList:");
MyIterator it1 = names.createIterator();
while (it1.hasNext()) {
    System.out.println(it1.next());
}

// Tree creation
TreeNode root = new TreeNode("A");
TreeNode b = new TreeNode("B");
TreeNode c = new TreeNode("C");
TreeNode d = new TreeNode("D");
TreeNode e = new TreeNode("E");

root.addChild(b);
root.addChild(c);
b.addChild(d);
b.addChild(e);

// Tree traversal
TreeCollection tree = new TreeCollection(root);

System.out.println("\nTraversing Tree (DFS):");
MyIterator it2 = tree.createIterator();
while (it2.hasNext()) {
    System.out.println(it2.next());
}
}
}
```

OUTPUT:

Traversing ArrayList:

Alice

Bob

Charlie

Traversing Tree (DFS):

A

B

D

E

C